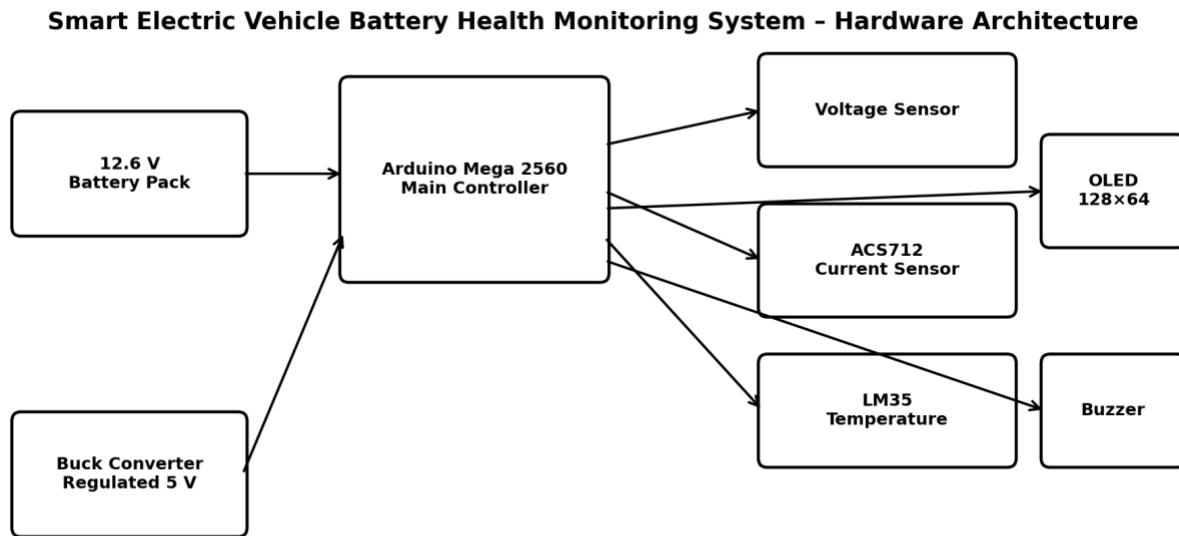


Phase 4 – Prototype Development Review (PDR) Deliverables

Prototype platform: Arduino Mega 2560. The system monitors battery voltage, current and temperature in real time, estimates State of Charge (SoC), shows values on a 128×64 I²C OLED, and provides an alert using a buzzer when programmed safety limits are exceeded.



Measures voltage, current and temperature; estimates SoC; displays status and gives an alert.

1. Working Prototype (Version 1)

The Working Prototype (Version 1) is the first complete physical implementation of the Smart Electric Vehicle Battery Health Monitoring System.

The prototype combines the Arduino Mega 2560, battery pack, voltage sensor, ACS712 current sensor, LM35 temperature sensor, OLED display, buzzer and regulated power supply.

During operation, the controller continuously reads the sensor values, processes the measurements and displays battery status. The first version is intended to prove that the complete measurement and alert functions work together before final enclosure or PCB development.

Main functions demonstrated:

- Real-time battery voltage measurement.
- Real-time current measurement.
- Battery temperature measurement.
- Estimated SoC calculation using the programmed voltage range.
- OLED display of measured values and status.
- Buzzer alert for abnormal or unsafe programmed conditions.
- Serial Monitor output for testing and debugging.

2. Embedded Software

The embedded software is the Arduino program developed in the Arduino IDE for the Mega 2560.

The program initializes the I²C OLED, analog sensor inputs and buzzer output. It then reads the voltage, current and temperature values at regular intervals.

The voltage reading is converted into battery voltage and an estimated SoC is calculated between the selected full and empty voltage limits.

The measured data are formatted and displayed on the OLED. Safety conditions are checked continuously, and the buzzer is activated when a programmed limit is crossed.

Software technologies:

- • Arduino IDE
- • Embedded C/C++
- • Wire library for I²C communication
- • Adafruit GFX and SSD1306 libraries for the OLED
- • Analog-to-digital conversion for sensor inputs
- • Serial communication for debugging

3. PCB Fabrication Status

For the current Version 1 prototype, the system is implemented on a breadboard/prototype wiring arrangement rather than a fabricated PCB.

Therefore, the PCB fabrication status can be reported as: "PCB not fabricated – breadboard prototype completed."

The PCB stage can be taken up after the circuit is electrically verified and the sensor readings are stable. A future PCB would place the controller interface, sensor connectors, power regulation and protection elements in a compact and reliable layout.

Recommended report statement:

"PCB fabrication is not applicable to Version 1. The circuit has been assembled and tested on a breadboard. PCB design will be considered after successful prototype validation."

4. Mechanical Assembly

The mechanical assembly refers to the physical arrangement and mounting of the battery, controller, sensors, display and wiring.

For Version 1, the components are arranged as a bench-top prototype. The Arduino Mega and sensors are kept accessible for testing and modification.

A final mechanical enclosure can be developed after electrical testing, with attention to ventilation, insulation, wire management, display visibility and secure mounting.

Version 1 status:

- • Breadboard-based assembly.
- • Components kept accessible for testing.
- • No final enclosure required at this stage.
- • Future work: compact enclosure and protected cable routing.

5. Hardware Integration

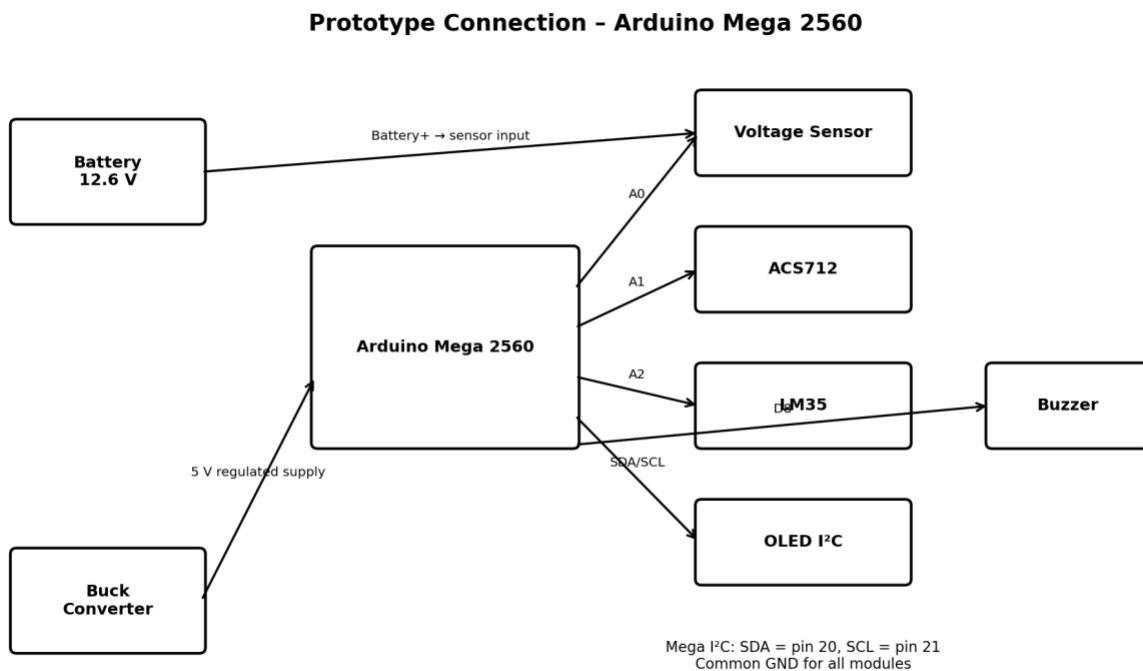
Hardware integration is the process of connecting all electronic modules so that they operate as one system. The battery provides the measurement source. The voltage sensor measures battery voltage, the ACS712 measures current, and the LM35 measures temperature.

The Arduino Mega 2560 receives these signals through analog inputs. The OLED communicates through I²C, while the buzzer is controlled through a digital output.

A regulated supply from the buck converter can be used to provide the required supply voltage to the controller and compatible modules. All grounds must be connected appropriately.

Example Mega 2560 interface mapping used in the prototype:

- Voltage sensor output → A0
- ACS712 output → A1
- LM35 output → A2
- OLED SDA → Mega pin 20
- OLED SCL → Mega pin 21
- Buzzer → D8
- Common GND between connected modules
- Regulated 5 V supply → Arduino Mega 5 V input/power arrangement as designed for the prototype



Illustrative connection diagram for the Phase 4 prototype; verify each module's voltage/current rating before powering.

6. Software Integration

Software integration connects the sensor acquisition, calculations, display and alert logic into one working firmware program.

First, the program initializes all hardware interfaces. Next, sensor values are read and converted into engineering values.

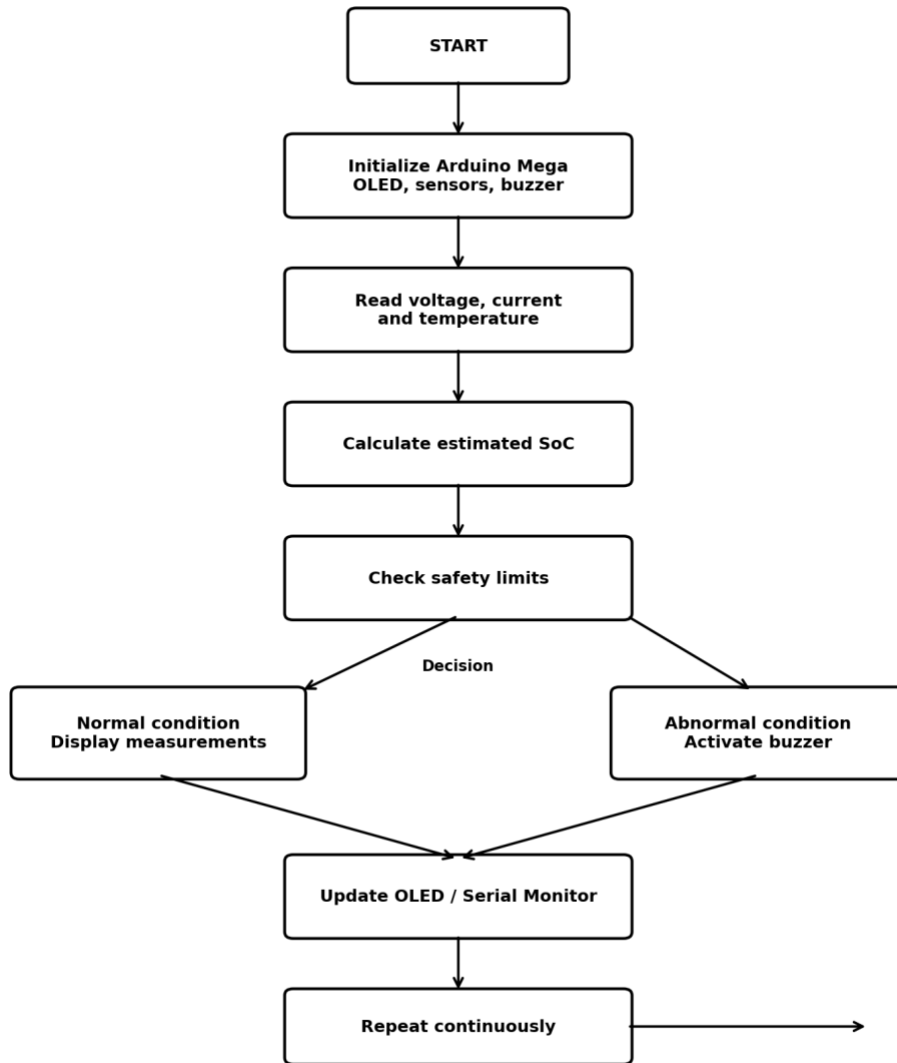
The SoC estimate is calculated from the selected battery voltage range. The processed values are then sent to the OLED and Serial Monitor.

Finally, the safety-check routine decides whether the buzzer should remain OFF or be activated. This loop repeats continuously, providing real-time monitoring.

Integrated software flow:

1. Initialize hardware and display.
2. Read voltage, current and temperature.
3. Convert raw ADC values to useful measurements.
4. Estimate battery SoC.
5. Check programmed safety thresholds.
6. Display measurements and status.
7. Activate/deactivate buzzer.
8. Repeat the monitoring cycle.

Software Flow - Smart EV Battery Monitoring



7. GitHub Repository Link

The GitHub repository is used to store the project source code and supporting project files.

For the Phase 4 submission, the repository should contain the final Version 1 Arduino program, README documentation, circuit/connection images, prototype photographs and relevant project reports.

The repository title used for this project is “smart-EV-battery-health-monitoring”. The actual repository URL should be pasted in the final submitted document.

GitHub submission model:

Repository Name: smart-EV-battery-health-monitoring

Repository URL: [https://github.com/uma993705-bot/SMART-EV-BATTERY-HEALTH-MONITORING-SYSTEM.git]

Recommended folders: /source-code, /images, /documents, /prototype

8. Weekly Progress Report

The weekly progress report records what the team completed during each week, the problems encountered, the solution applied and the next planned activity.

It provides evidence that the prototype was developed systematically rather than assembled only at the end of the project.

Suggested weekly format:

Week 1 – Finalized components and circuit architecture.

Week 2 – Connected Arduino Mega and tested individual sensors.

Week 3 – Integrated OLED display and verified real-time readings.

Week 4 – Added SoC calculation and safety alert logic.

Week 5 – Integrated the complete Version 1 prototype and performed testing.

Week 6 – Corrected wiring/software issues and prepared PDR evidence.

9. Submission – GitHub Repository

Submit the project GitHub repository containing the source code and supporting documentation. The README should include the project title, objective, hardware list, working principle, connection details, software requirements and prototype photographs.

Checklist:

- ☐ Repository is accessible.
- ☐ Source code is uploaded.
- ☐ README is updated.
- ☐ Circuit/prototype images are included.
- ☐ Project documentation is included.

10. Submission – Prototype Images

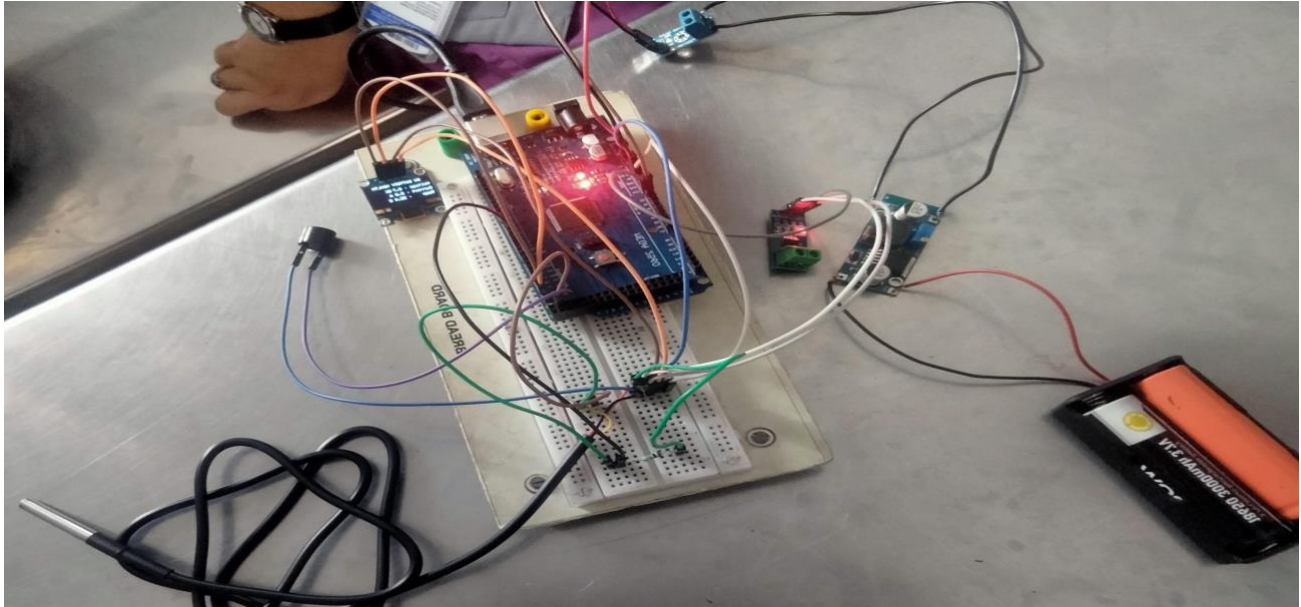
Prototype images should clearly show the complete Version 1 hardware. Include at least one overall photograph and additional close-up photographs of the Arduino Mega, sensors, OLED and power/battery connections.

Images should be clear enough for a reviewer to identify the main modules and wiring.

Recommended photographs:

1. Full prototype view.
2. Arduino Mega and sensor connections.

3. OLED displaying measurements.
4. Battery/voltage-sensor/current-sensor arrangement.
5. Buzzer and power-supply arrangement.



11. Submission – Prototype Video

The prototype video should demonstrate the actual working sequence. Start with a short introduction of the project, then show the hardware and finally demonstrate the live readings on the OLED.

The video should show how the system responds when a programmed monitoring condition is reached.

- • Project title and objective.
- • Show complete hardware setup.
- • Power ON the prototype.
- • Show voltage, current and temperature readings.
- • Show estimated SoC/status.
- • Demonstrate the alert condition if safely available.
- • End with the working prototype view.

12. Submission – Source Code

The source code submitted for PDR should be the same tested Version 1 firmware used in the physical prototype.

Add clear comments for sensor pins, OLED settings, SoC limits, safety thresholds and main program sections. Avoid uploading unrelated test programs as the main submission file.

Source-code structure:

- • Library includes

- • OLED configuration
- • Sensor pin definitions
- • Battery/SoC constants
- • setup() initialization
- • Sensor reading functions
- • SoC calculation
- • Display function
- • Safety/buzzer logic
- • loop() monitoring cycle

13. Submission – Weekly Progress Report

Attach the weekly report as a document or PDF and, where appropriate, keep the same report in the GitHub repository.

Each entry should include date/week, completed work, test result, issue, corrective action and next step.

Final Phase 4 PDR Submission Checklist

PDR Deliverable	Status / Evidence to Attach
Working Prototype (Version 1)	Complete hardware photograph + live demonstration
Embedded Software	Tested Arduino Mega source code
PCB Fabrication Status	Breadboard prototype; PCB not fabricated for Version 1
Mechanical Assembly	Bench-top/breadboard assembly photographs
Hardware Integration	Connection diagram + prototype photograph
Software Integration	Flowchart + working OLED/Serial output
GitHub Repository Link	Repository URL
Weekly Progress Report	Weekly table/report
Prototype Images	Clear overall and close-up images
Prototype Video	1–3 minute working demonstration
Source Code	Final tested Version 1 program

Important: Replace the GitHub URL placeholder and insert your real prototype photographs/video link before final submission.